

Software Requirements Specification

For

Student information system

Prepared by

Jomana Emad Ragab Sakr 320240100

Mariam Mohamed Ahmed Ayob 320240091

Asma Shokr 320240092

Nada Ashraf 320240088

Organization

Egypt-Japan university of science and technology

Date of creation

11-11-2025

1. Introduction	4
1.1 Purpose	4
1.2 Product Scope.....	4
1.3 Overview.....	5
2. Overall description	6
2.1 product perspective.....	6
2.2 product function.....	6
2.3 User Classes and Characteristics	7
2.4 Operating Environment	8
2.5 Design and Implementation Constraints	8
2.6 User Documentation.....	9
3. External interface requirements.....	11
3.1 User Interfaces.....	11
3.2 Hardware Interfaces	11
3.3 Software Interfaces.....	11
3.4 Communications Interfaces	11
4. System features	12
4.1 System Feature	12
Add Student.....	12
5. Non-Functional Requirements.....	15
5.1 Performance Requirements.....	15

1. Introduction

1.1 Purpose

The purpose for this project is to define the requirements for university management system

this system is intended to automate, and manage operations within a university, such as:

- Student registration and course enrollment
- Professor management
- Course and department administration
- Exam scheduling and grading
- Library operations
- Attendance tracking
- User authentication and role-based access

This SRS is targeted at:

- Developers
- Project supervisors
- University staff involved in system design
- Testers verifying system functionality

1.2 Product Scope

The Student Information System (SIS) is a software application designed to manage and organize all student-related data within an educational institution. The system enables administrators, faculty members, and authorized staff to efficiently access, update, and track student records, including personal information, course enrollments, grades, schedules, and academic performance.

The primary purpose of the SIS is to replace manual or paper-based processes with an integrated digital solution that improves accuracy, reduces administrative workload, and enhances decision-making. By centralizing student data in one platform, the system ensures faster access to information, minimizes errors, and supports better communication between departments.

The SIS supports the institution's strategic goals by contributing to digital transformation, improving operational efficiency, and enhancing the overall student experience. It aligns with the organization's goals of providing reliable academic services, ensuring data integrity, and streamlining administrative workflows.

1.3 Overview

This SRS includes:

- System description
- Product functions
- User characteristics
- Functional and non-functional requirements
- Class diagrams & interactions
- System constraints

2. Overall description

2.1 product perspective

This is the first version of the Student Information System. It is a web-based platform designed to store and manage (specific users' data) students' data in an organized way. The system supports a large number of users and allows each user to access the information they need based on their role.

2.2 product function

1. Student Management

- Add, update, and manage student information.
- Register new students.
- Enroll students in courses.
- Remove students from courses.
- View student course enrollments.
- Track and view student attendance.
- Calculate attendance percentage.

2. Professor Management

- Add, update, and manage professor information.
- Register professors.
- Assign professors to courses.
- View professor-related course and department information.

3. Course Management

- Add and update courses.
- Enroll students in courses.
- Remove students from courses.
- Show list of students enrolled in each course.
- Manage course attendance.

4. Department Management

- Add departments.
- Assign students and professors to departments.
- Display department information.

5. Administration Management

- Register administrators.
- View administrator information.
- Manage user roles and permissions.

6. Classroom & Scheduling System

- Add classrooms with capacities.
- Create course schedules.
- Update existing schedules.
- View all schedules.

7. Exam Management

- Add exams.
- Schedule exam times and rooms.
- Record student exam results.
- View exam results by student or course.

8. Library Management (Physical Library Only)

- Add a library.
- Add books to the library.
- Register students for library membership.
- Borrow and return books.
- Search for books by title, author, or ISBN.

9. Attendance Management

- Record daily course attendance.
- View attendance by student or by course.
- Update previously recorded attendance.

10. User Authentication & Access

- Login and logout functionality.
- Secure role-based dashboards (Student, Professor, Admin).
- Register new system users.

11. System Controls

- Exit and safely terminate the system.

2.3 User Classes and Characteristics

Primary stakeholders and user classes:

Administrator:

- Add/remove students
- manage courses
- assign professors
- update attendance

professor:

- add students grades
- update and record students attendance

students:

- View grades
- View attendance

- borrow library books

system user:

- Login/Logout
- view dashboard

2.4 Operating Environment

- Web-based (HTML, CSS, JS)
- Backend using Python
- Database: MySQL, PostgreSQL, or SQL Server
- Runs on Windows

2.5 Design and Implementation Constraints

1 Programming Language and Framework

- The system is implemented using **Python 3.x**, which constrains developers to Python syntax, libraries, and runtime environment.
 - Use of object-oriented programming (OOP) principles is mandatory, as classes like Student, Admin, Library, and Attendance are central to system functionality.
 - Abstract base classes (ABCs) must be used for features requiring extensibility, such as ScheduleBase and Exam.
- Hardware Limitations**

- The system is designed to run on standard desktop or laptop computers with at least **4 GB RAM** and **1 GHz CPU**, limiting high-performance parallel computations.
- Memory consumption should be minimized, as all records (students, courses, schedules, attendance, exams) are stored in memory using Python lists and dictionaries, not in a database.

Data Storage and Databases

- Currently, **persistent storage is not implemented**; all data is in-memory. This constrains developers from designing features that require persistent data across sessions.
- Future expansion may require integrating a relational database (e.g., MySQL, SQLite) or NoSQL database (e.g., MongoDB).

User Interface Constraints

- The system uses a **text-based command-line interface (CLI)**.
- Constraints include limited visual representation, no drag-and-drop functionality, and reliance on user input via keyboard.

Security Constraints

- User authentication is handled via **email and password**, but passwords are stored in

plain text in memory, limiting security.

- The system must enforce **role-based access control** for features like attendance modification and schedule updates.

- Sensitive operations (e.g., attendance updates, exam results) are restricted to authorized roles (admin, professor).

Communication and Integration

- The system currently does **not integrate with external applications** (e.g., email servers, online library catalogs), limiting automated notifications or data import/export. • Any integration in the future must adhere to standard communication protocols (e.g., HTTP, REST APIs).

Parallel Operations

- The system is **single-threaded**, and only one user session can be active at a time.
- Multi-user simultaneous access or concurrent modifications are not supported in the current implementation.

Design Conventions and Standards

- Naming conventions for classes, methods, and variables must remain consistent (e.g., CamelCase for classes, snake_case for methods).
- Exception handling is required for invalid inputs (e.g., date formats, invalid IDs, role restrictions).

2.6 User Documentation

Student User Manual

Explains how students can:

- Login/logout and access their dashboard.
- View their courses and schedules.
- Register for courses.
- Check exam schedules and results.
- Access the library, borrow and return books.
- View attendance and calculate attendance percentage.

Delivered in **PDF** and **HTML** formats.

Professor User Manual

Explains how professors can:

- Login/logout and access their dashboard.
- Manage their courses.
- Record student attendance.
- Schedule and record exam results.
- View course enrollment and department info.

Delivered in **PDF** and **HTML** formats.

Administrator User Manual

Explains how administrators can:

- Login/logout and access their dashboard.

- Add and manage students, professors, courses, classrooms, and departments.
- Create and update schedules.
- Add and manage library resources.
- Register users and assign roles.

Delivered in **PDF** formats.

Quick Start Guide

A concise guide for all users to quickly perform basic tasks, such as logging in, registering for courses, or recording attendance.

Delivered in **PDF** format.

3. External interface requirements

3.1 User Interfaces

- Responsive web interface
- Dashboard for each user role
- Forms for data entry
- Tables for displaying information

3.2 Hardware Interfaces

There is no hardware interface

3.3 Software Interfaces

- Database server (MySQL/PostgreSQL)
- Email/SMS notification API (optional)

3.4 Communications Interfaces

The university management system communication capabilities mainly for web access

Web communication

1. The platform must support standard HTTP/HTTPS protocols for: Page requests
 - i. Course retrieval
 - ii. Media loading
2. Uses SMTP or provider-specific API.

4. System features

4.1 System Feature

Add Student

Description and Priority

Adds a new student to the system by providing ID, name, major, and email.

Priority: High

Stimulus/Response Sequences

- 1- User selects "Add Student" from the menu.
- 2-System prompts for student name, ID, major, and email.
- 3-User inputs details.
- 4-System creates a new **Student** object and adds it to the student list.
- 5-The system displays "Student added successfully!"

Functional Requirements

REQ-1: The system shall prompt the user for the student's name, ID, major, and email.

REQ-2: The system shall create a **Student** object using the provided information.

REQ-3: The system shall add the new student to the student list.

REQ-4: The system shall display a confirmation message.

REQ-5: The system shall handle duplicate IDs gracefully

Add Professor

Description and Priority

Adds a new professor with ID, name, department, contact info, and email.

Priority: High

Stimulus/Response Sequences

- 1-User selects "Add Professor".
- 2-System prompts for professor details.
- 3-User inputs data.
- 4-System creates a Professor object and adds it to the professors list.
- 5-System updates the relevant department if exists.
- 6-System confirms addition.

Functional Requirements

- **REQ-1:** Prompt for professor's name, ID, department, contact info, and email.
- **REQ-2:** Create a Professor object with input data.

- **REQ-3:** Add professor to the professors list.
- **REQ-4:** Link professor to their department if department exists.
- **REQ-5:** Display confirmation message.

Add Course

Description and Priority

Adds a new course and assigns a professor to it.

Priority: High

Stimulus/Response Sequences

- 1-User selects "Add Course".
- 2-System prompts for course name, ID, department, credits, and professor selection.
- 3-User enters details and chooses a professor.
- 4-System creates Course object and links it to the professor.
- 5-Course is added to the courses list and department.
- 6-System confirms addition.

Functional Requirements

- REQ-1:** Prompt for course details and professor selection.
- REQ-2:** Validate professor selection.
- REQ-3:** Create Course object.
- REQ-4:** Add course to courses list and professor's courses.
- REQ-5:** Confirm course addition.

Enroll Student in Course

Description and Priority

Enrolls a student in a course by updating both student and course records.

Priority: High

Stimulus/Response Sequences

- 1-User selects "Enroll Student in Course".
- 2-System prompts for student ID and course ID.
- 3-System validates inputs.
- 4-If valid, system adds course to student and student to course.
- 5-System displays confirmation or error message.

Functional Requirements

- REQ-1:** Validate student ID and course ID.
- REQ-2:** Enroll student in course if not already enrolled.
- REQ-3:** Update student's courses list and course's enrolled students.
- REQ-4:** Display confirmation message.

Remove Student from Course

Description and Priority

Removes a student from a course.

Priority: Medium

Stimulus/Response Sequences

- 1-User selects "Remove Student from Course".
- 2-System prompts for student ID and course ID.
- 3-System validates inputs.
- 4-If valid, system removes student from course and course from student.
- 5-System displays confirmation or error message.

4.5.3 Functional Requirements

- REQ-1:** Validate student and course existence.
- REQ-2:** Remove student from course enrollment list.
- REQ-3:** Remove course from student's courses list
- REQ-4:** Display appropriate message.

5. Non-Functional Requirements

5.1 Performance Requirements

PERFORMANCE:

The web page should load each page within 2 seconds and provide a real time update

Security:

Ensure proper protection for passwords and students data

Availability:

The software should be available at all times

Usability:

UI easy to use for students, professors and staff

Maintainability:

The systems architecture should be modular so it will be easy to update and develop for developers

Compatibility:

The system shall remain fully compatible with the latest stable versions of major web browsers, including **Google Chrome, Mozilla Firefox, Microsoft Edge, and Safari.**

Reliability:

The system should be reliable, maintaining consistent performance across features without crashes or data loss